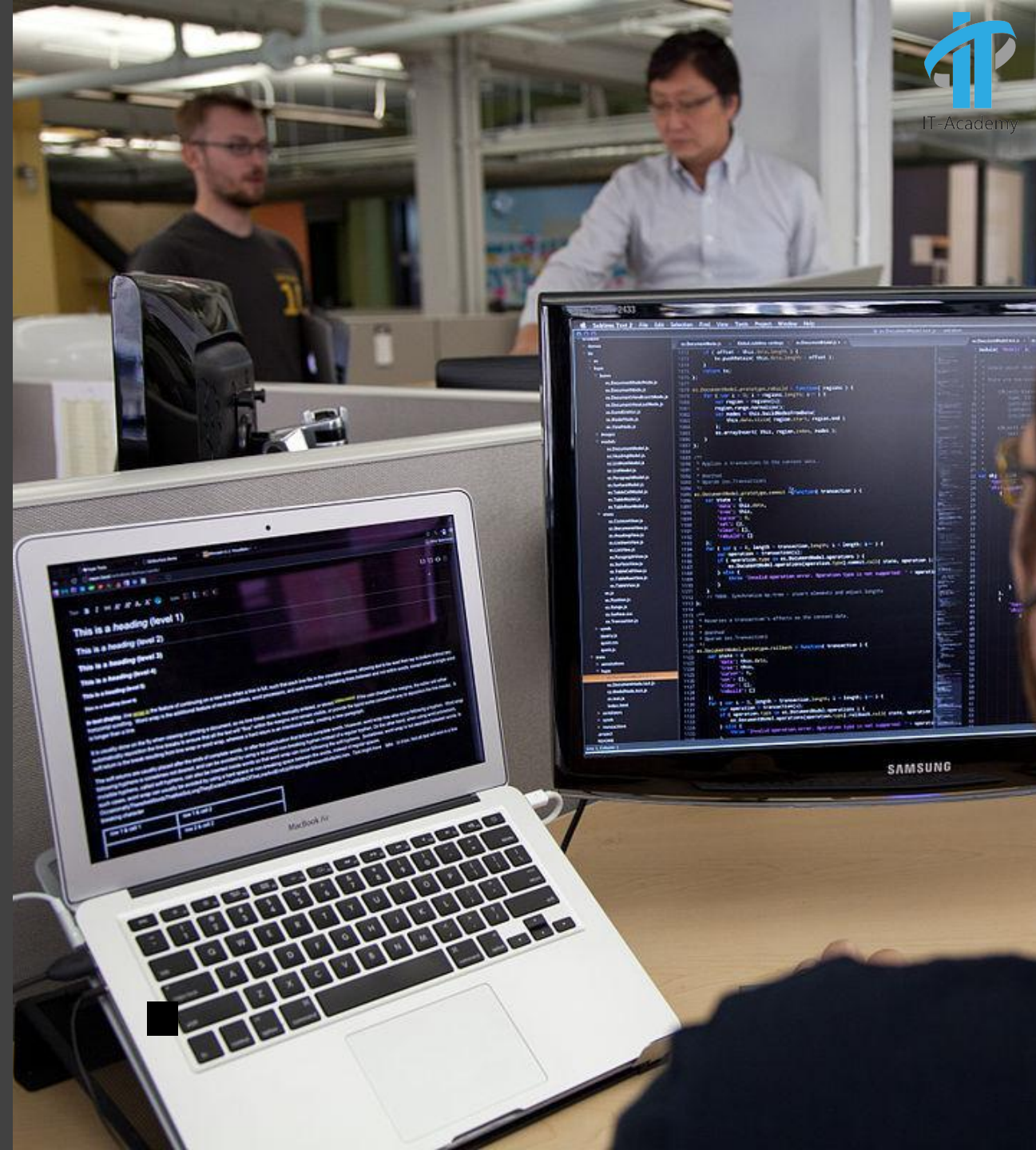
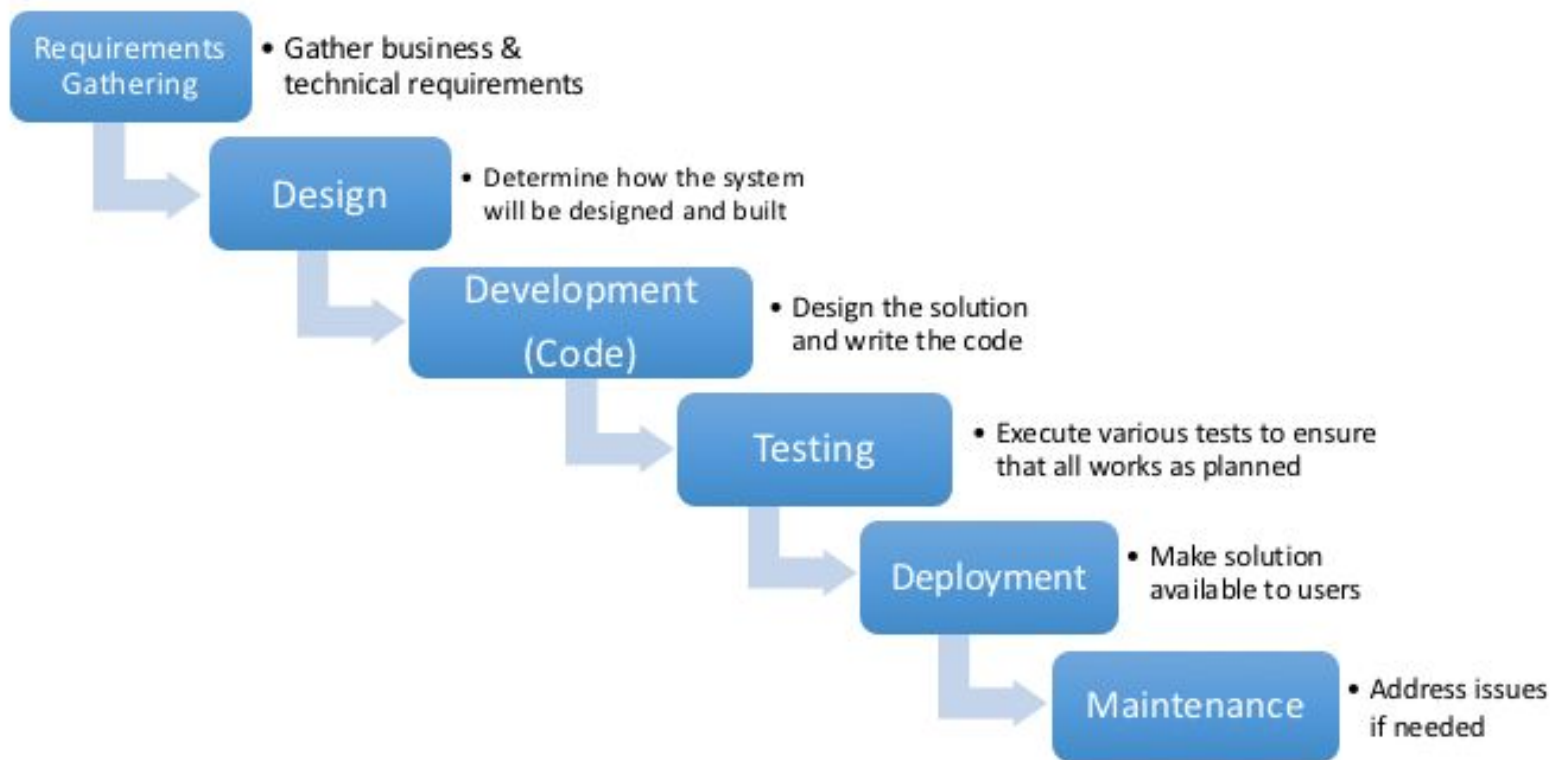


Software Development Life Cycle SDLC Жизненный цикл ПО



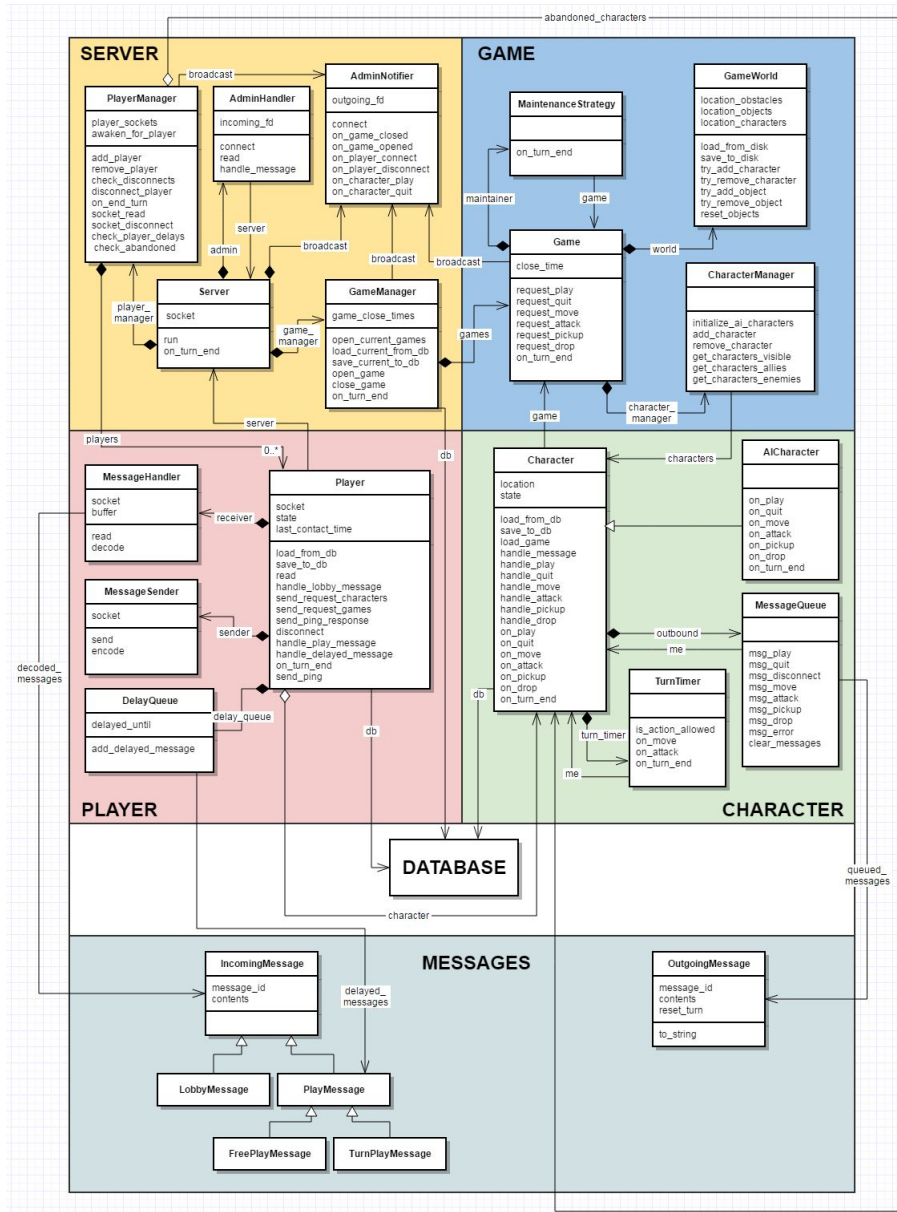
Overview of Software Development Life Cycle (SDLC)



Жизненный цикл программного обеспечения (также называемый циклом разработки) – это условная схема, включающая отдельные этапы, которые представляют стадии процесса создания ПО. При этом на каждом этапе выполняются разные действия.

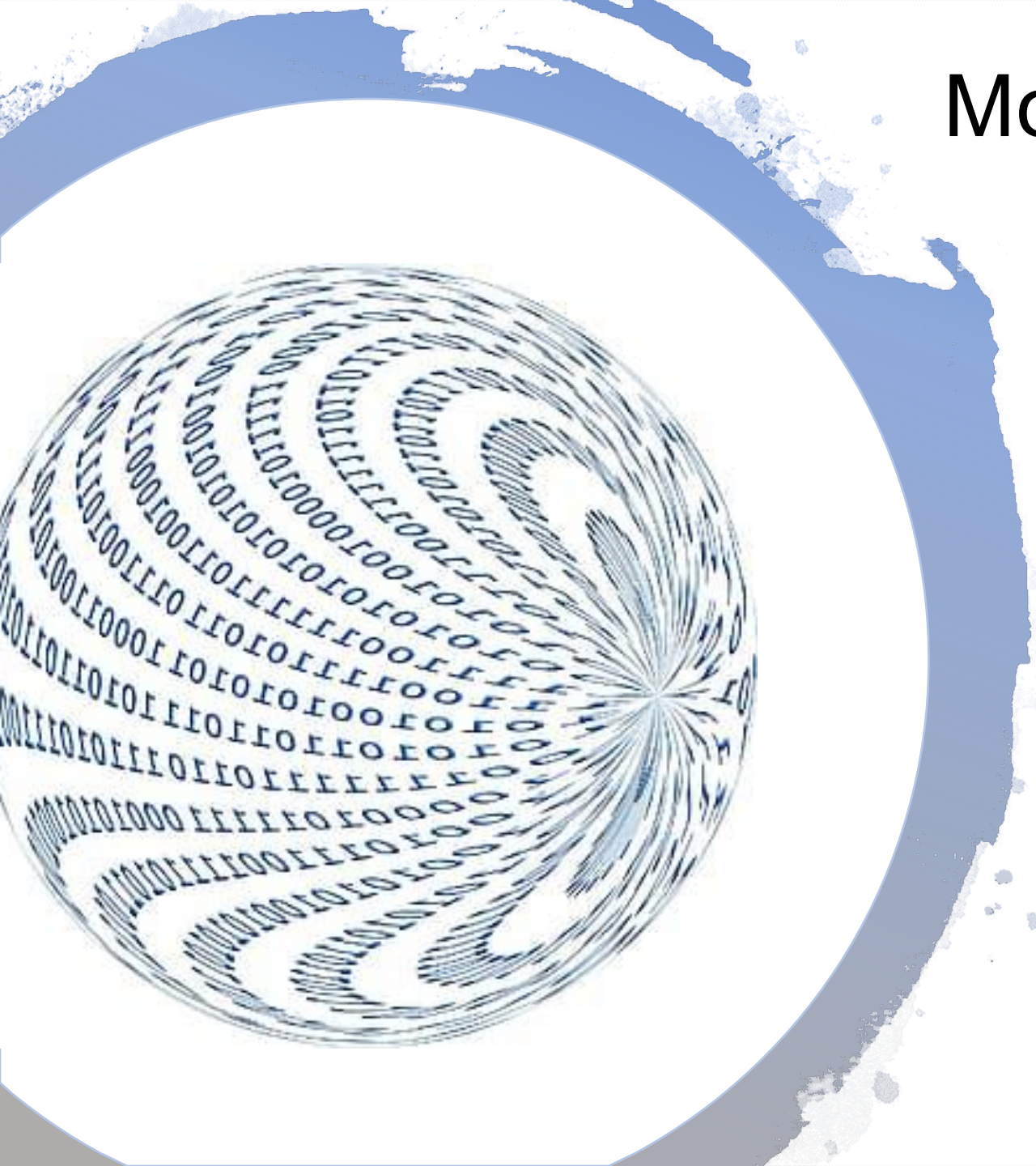
Стандарт ГОСТ 34.601-90

ISO/IEC 12207:2008 Systems and software engineering



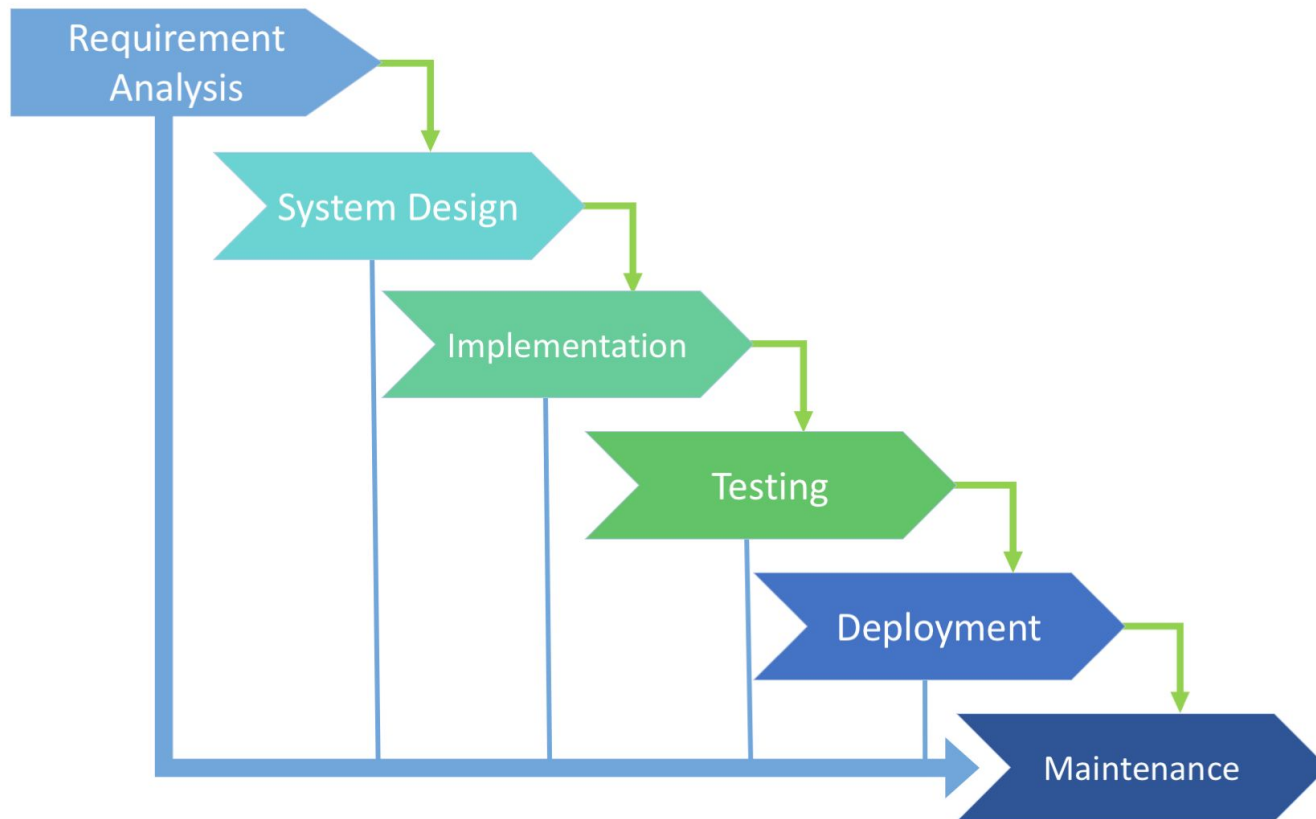
Модели SDLC

- ❖ Модель водопада
- ❖ Итерационная модель
- ❖ Спиральная модель
- ❖ V-модель
- ❖ RAD



SDLC – модель водопада

- ❖ линейно-последовательная модель
- ❖ самый ранний
- ❖ фазы не перекрываются



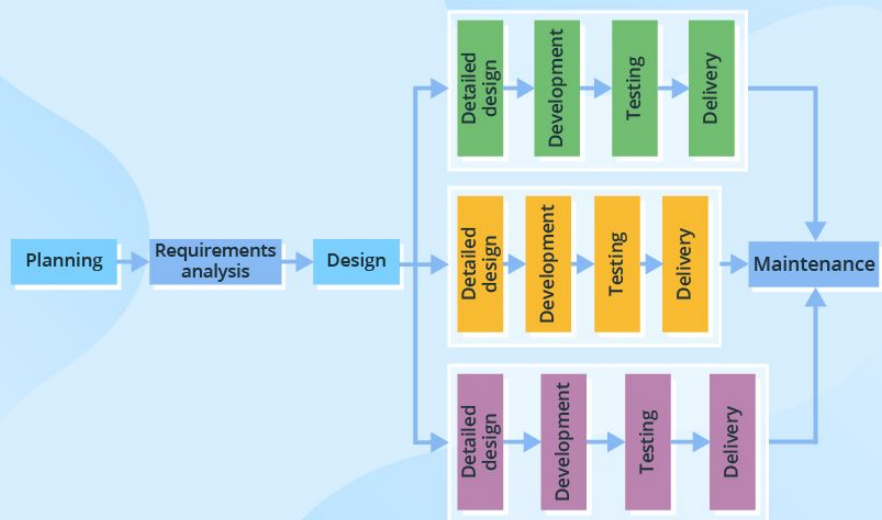
Waterfall SDLC - рискованная. Используйте, где возможно, решения:

- модульная поставка
- тесная работа DEV & QA

SDLC – итеративная и инкрементальная модели

- ❖ начинаем с небольшого набора **требований** и итеративно расширяем
- ❖ создаем новую **версию ПО** в конце каждой итерации
- ❖ разрабатываем систему с помощью повторяющихся циклов (итеративно) и меньшими порциями за один раз (постепенно).

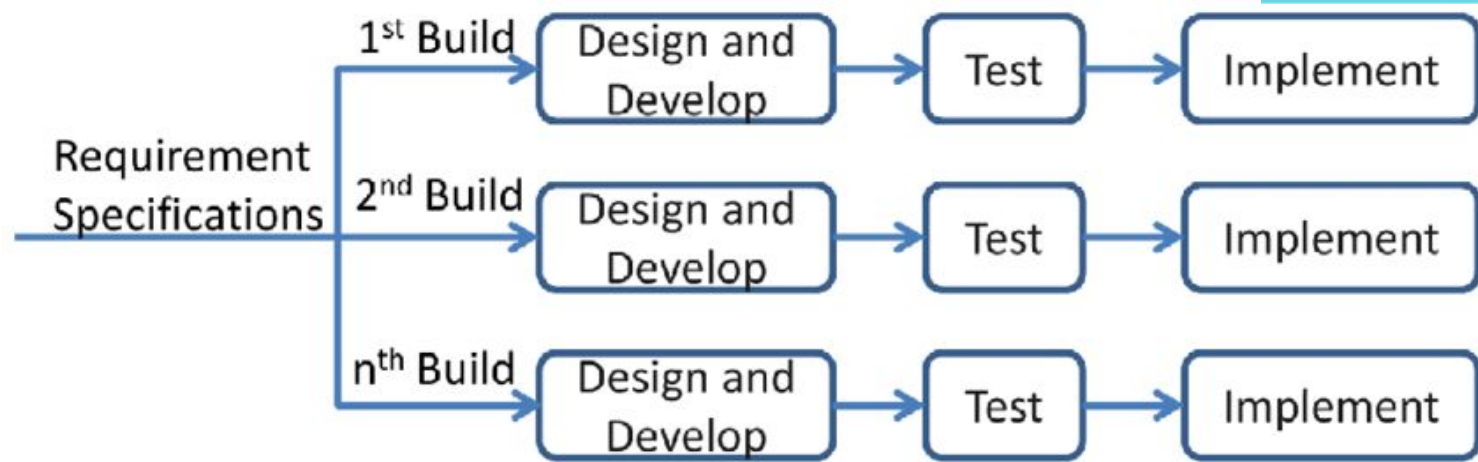
INCREMENTAL MODEL

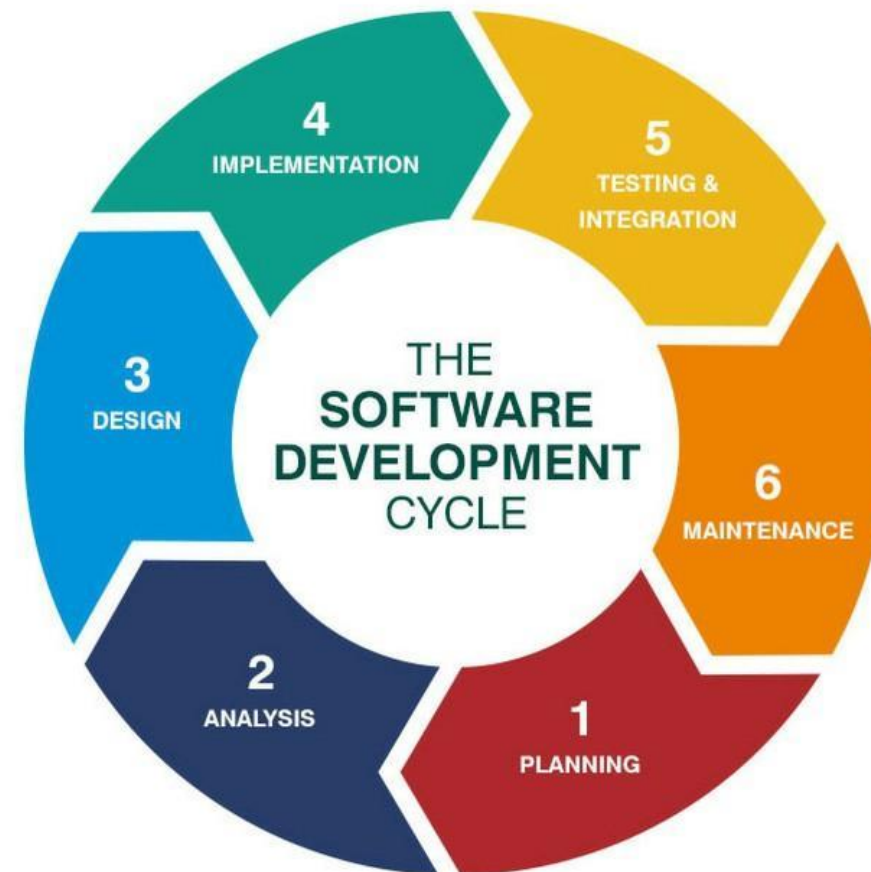
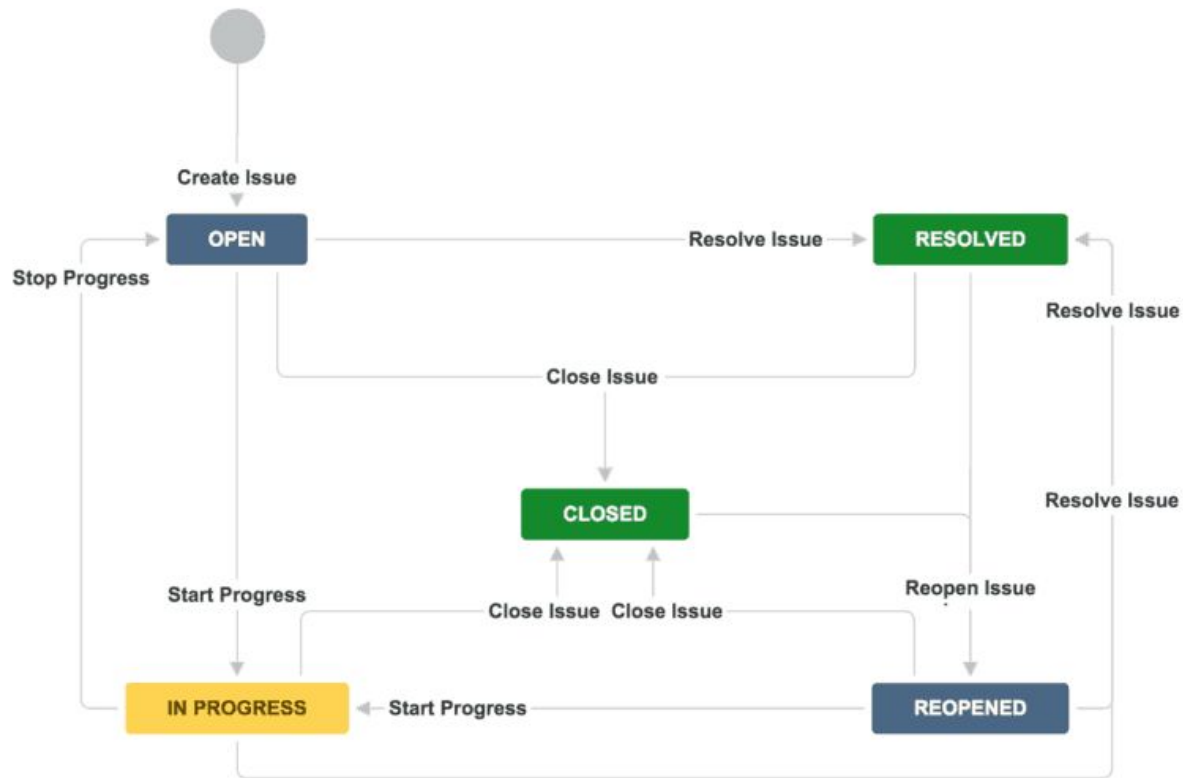


Инкрементная модель



Итеративная модель

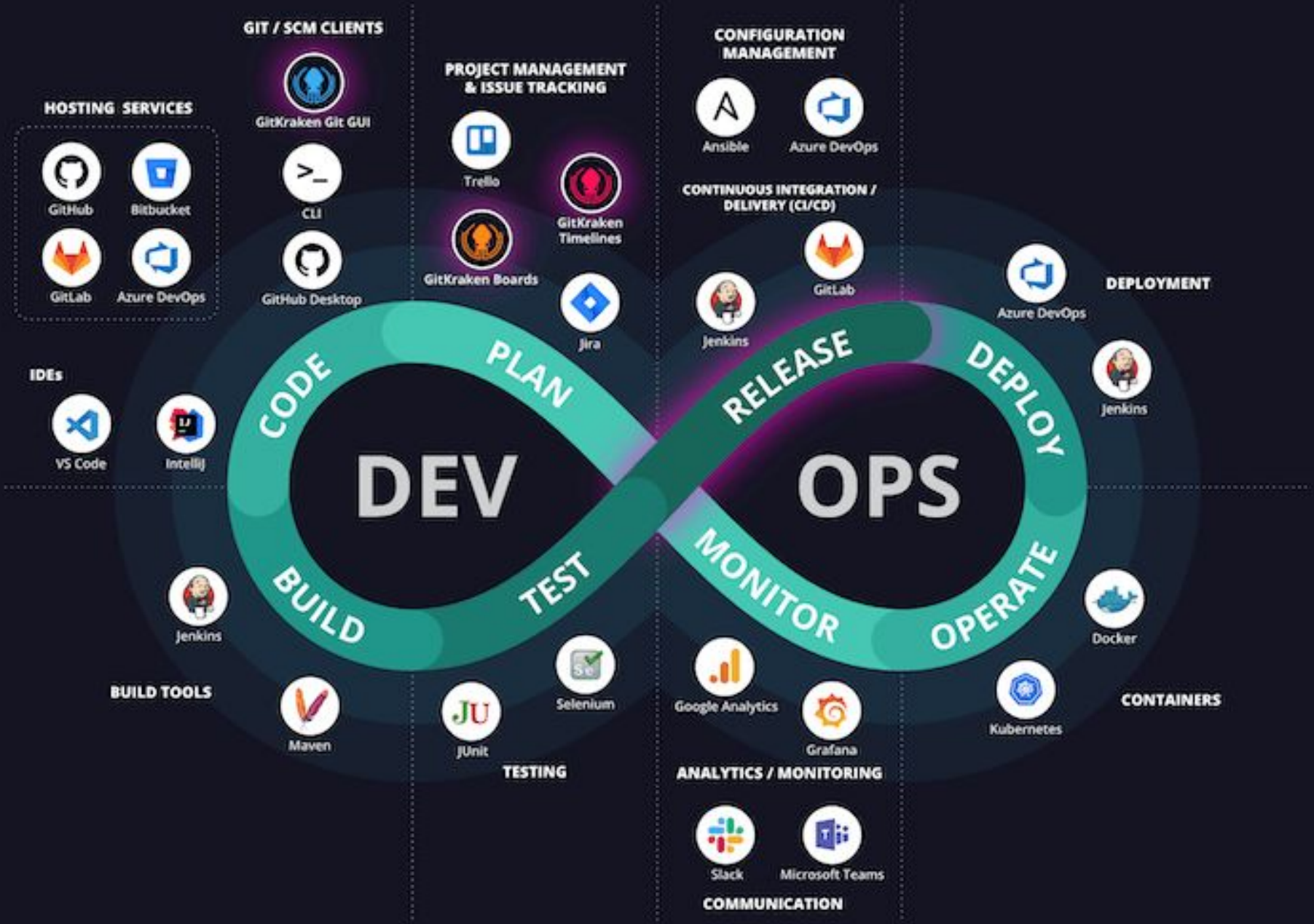




Проводя параллели между SDLC и Workflow задачи, увидим, что жизненный путь задачи во многом проходит те же этапы.



DevOps

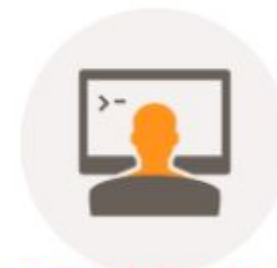
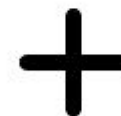


DevOps - культура на стыке
разработки и операций.

What is DevOps?



Developers & Testers



IT Operations

A “**build**” is a developed application that is given by development team to the **software** testing team.

A “**release**” is an official launch of the application for the customers.

A **build** when tested and certified by the **software** testing team is provided to the customers as “**release**”

Deployment management

Moving, installing and configuring of new/changed **software**, hardware and documentation or any other service component to target environment.

Release management

Making new/changed services available for public use



Deployment



Ключевые этапы процесса управления релизами:

1. планирование;
2. сборка релиза (построение релиза);
3. тестирование;
4. развертывание;
5. контроль качества (анализ после релиза).

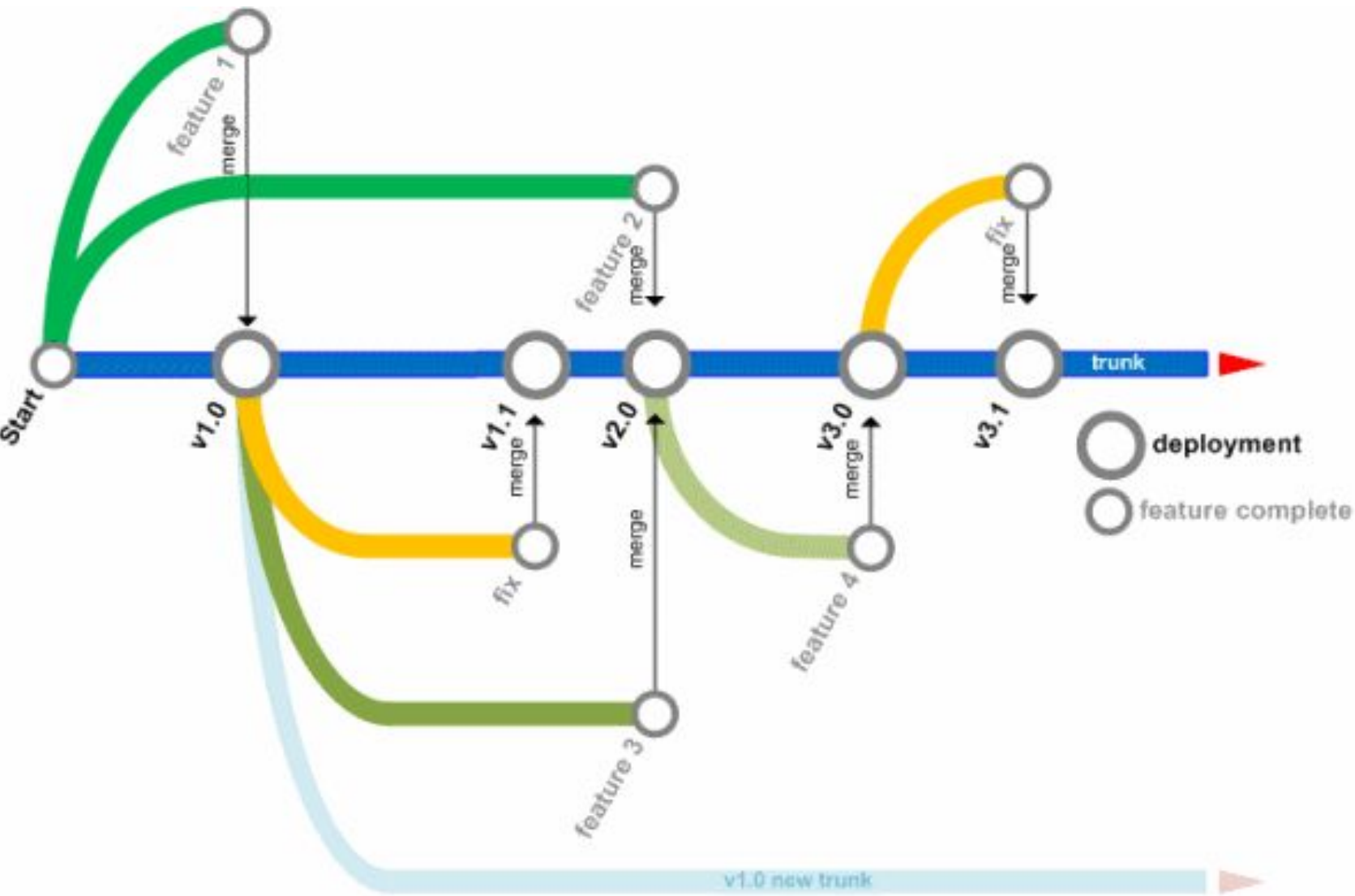
Software environments

Software Environment is the collection of hardware and software tools a system developer uses to build software systems.

Usually we have the following:

- A Development Environment;
- A Staging (PreProd) Environment; and
- A Production Environment.

Version Control



GitHub

GitHub — **веб-сервис** для размещения репозитория и совместной разработки проектов.

Проект в GitHub хранится в **репозитории** (*repository*) — коллекции всех изменений создаваемого кода. Каждый разработчик клонирует репозиторий первоначального создателя проекта.

Внутри репозитория изменения кода хранятся в виде **веток** и **КОММИТОВ**.



Талисманом GitHub выбран осьмикот

Коммит (commit) — основной объект разработки, по сути сохранение, фиксация в репозитории изменений в программном коде.

У каждого коммита есть атрибуты: имя, дата создания, автор и комментарии к текущей версии.

Ветка (branch) — указатель на коммит с определенными изменениями. Ветвление означает, что вы отклоняетесь от основной линии разработки и начинаете работать, не вмешиваясь в основную линию.

Основной веткой проекта считается ветка **main** или **master** — разработчики создают новые ветки на ее основе. Также можно создать неограниченное количество веток, чтобы вносить новые изменения, не мешая основному проекту.

Разработчики делают параллельные изменения кода. По окончании работы ветки каждого из них можно объединить в одну, чтобы создать коммит со всеми внесенными разработчиками изменениями.

Для этого в Git используют функцию `pull request` (PR).

`Pull request` — это заявка на слияние кода из разных веток. В процессе слияния Git создаст коммит и покажет все изменения в файле кода: добавленные до разветвления строки подсвечиваются зеленым цветом, удаленные — красным. Так каждый из разработчиков и менеджер проекта увидят, что произошло с кодом после совместной работы над коммитом.

Перед окончательным слиянием (`merge`) все разработчики должны просмотреть изменения кода (`code review`) и принять их.

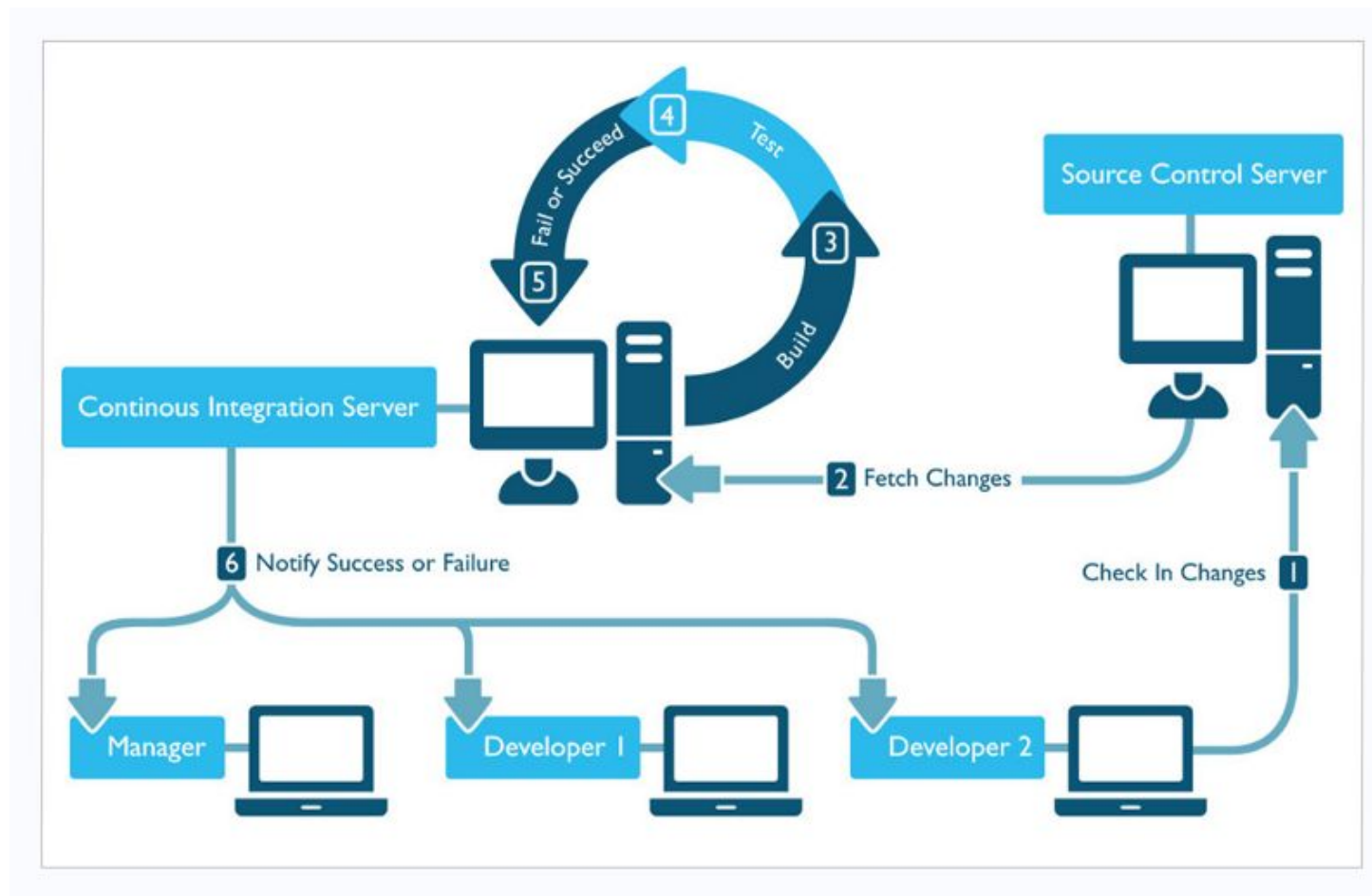
Ревью кода ([code review](#)) — процесс обсуждения изменений кода после совместного создания коммита и перед окончательным слиянием. В ревью разработчики оставляют комментарии к строкам с измененным кодом, а в случае ошибок или упущенных моментов предлагают решения по улучшению кода.

После ревью разработчики должны закрыть комментарии и принять предлагаемые изменения (функция [approve](#)). Git объединит ветки с помощью функции [merge](#) и перенесет созданный коммит в основную ветку `main`. В истории коммитов останется отметка о проведенном слиянии веток.

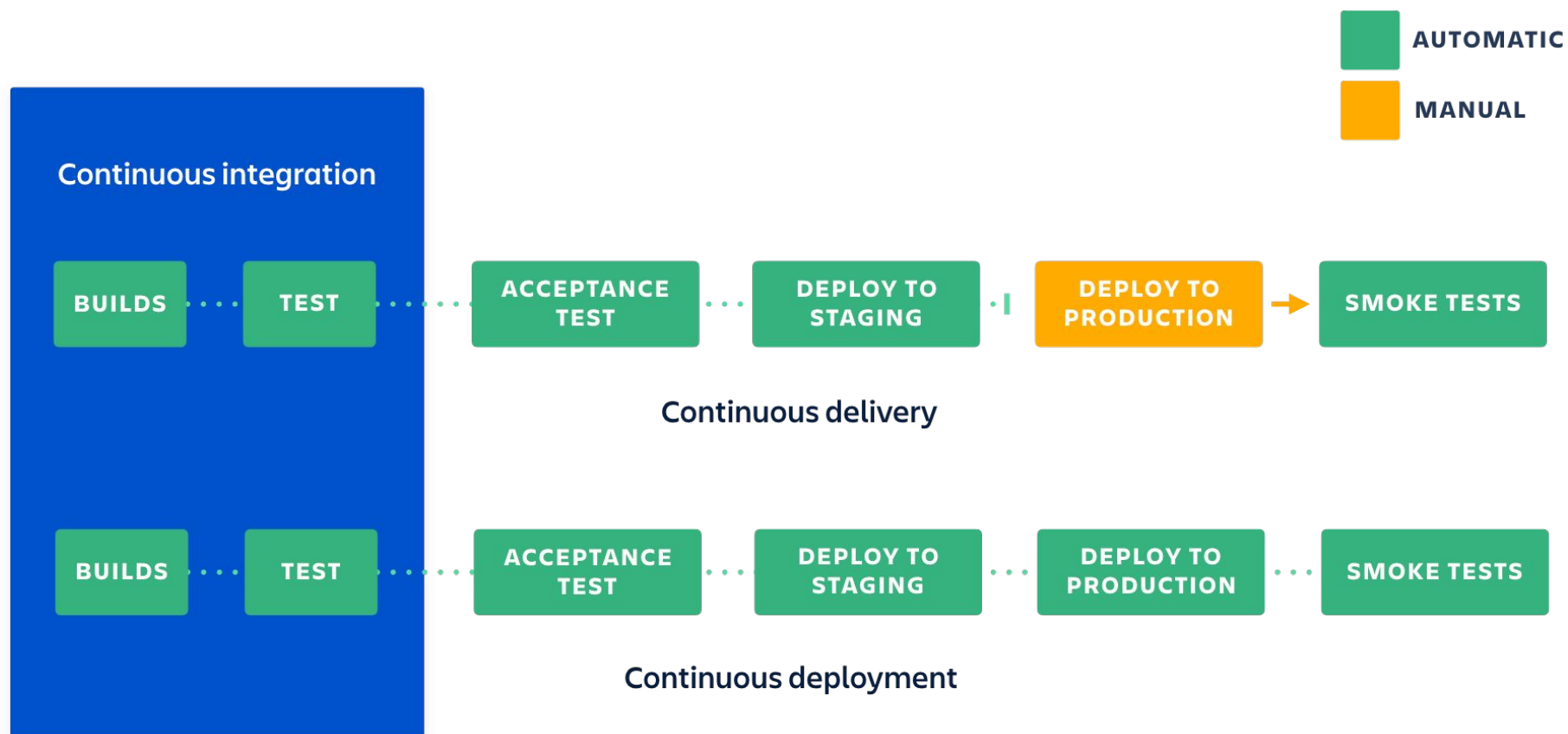


Continuous Integration Непрерывная интеграция

практика разработки программного обеспечения, которая заключается в слиянии рабочих копий в общую основную ветвь разработки несколько раз в день и выполнении частых автоматизированных сборок проекта



Continuous Delivery/Deployment



Практика (в группах)

1. Разработайте Workflow для вашего проекта (cases)

2. Продумайте DoD для каждого этапа в workflow

Что сделать дома

- почитать статью Gitflow
- почитать статью DevOps fundamentals
- почитать статью CI/CD